

计图第七次实验总结

学院：计算机学院 姓名：林晖鹏 学号：2312966

一、实验内容

- 使用MindSpore框架进行深度学习网络DCGAN搭建，利用动漫头像数据集进行模型训练、图像生成；
- 通过本实验了解深度学习任务开发的具体流程，包括：**数据处理、创建网络、连接网络、损失函数、优化器、模型训练及图像生成**；
- 了解经典神经网络DCGAN的结构特点和创新设计，以及深度学习网络搭建的重要概念。

二、DCGAN

DCGAN的核心逻辑延续了GAN的“对抗训练”思想，即通过生成器（Generator，G）与判别器（Discriminator，D）的相互对抗、相互优化，最终使生成器能够学习到真实数据的分布，生成与真实数据相似的样本。DCGAN对生成器和判别器均进行了针对性的卷积结构优化，摒弃了传统GAN中常用的全连接层，完全采用卷积操作（生成器含转置卷积）实现特征提取与样本生成，

三、主要实验流程

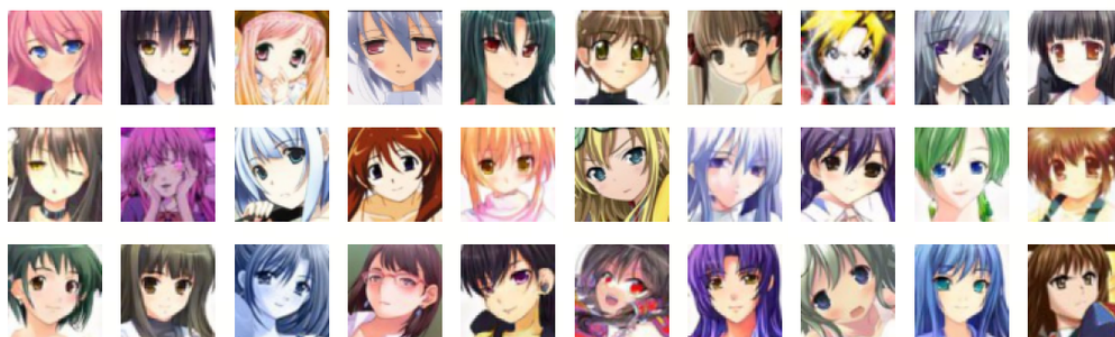
1. 数据增强

数据加载就不介绍了，这里额外对数据进行了增强，除了裁剪、放缩转换维度之外，主要是通过除以 255，将像素值线性缩放到 `[0, 1]` 区间，消除像素值量级差异对模型训练的影响。

代码块

```
1      # 数据增强操作
2      transforms = [
3          vision.Resize(image_size),
4          vision.CenterCrop(image_size),
5          vision.HWC2CHW(),
6          lambda x: ((x / 255).astype("float32"))
7      ]
```

这里放一组原始训练数据可视化，可以看到还是比较清晰的，但是后续的结果总是有点模糊甚至有伪影。



原始数据可视化

2. 网络构建

生成器 (G)

生成器的核心功能是将一个低维的随机噪声向量（通常为100维）映射为高分辨率的图像样本。DCGAN生成器采用“转置卷积+批量归一化+ReLU激活”的组合结构：

代码块

```
1 self.generator = nn.SequentialCell(
2     nn.Conv2dTranspose(nz, ngf * 8, 4, 1, 'valid', weight_init=weight_init),
3     nn.BatchNorm2d(ngf * 8, gamma_init=gamma_init),
4     nn.ReLU(),
5     nn.Conv2dTranspose(ngf * 8, ngf * 4, 4, 2, 'pad', 1,
6     weight_init=weight_init),
7     nn.BatchNorm2d(ngf * 4, gamma_init=gamma_init),
8     nn.ReLU(),
9     nn.Conv2dTranspose(ngf * 4, ngf * 2, 4, 2, 'pad', 1,
10    weight_init=weight_init),
11    nn.BatchNorm2d(ngf * 2, gamma_init=gamma_init),
12    nn.ReLU(),
13    nn.Conv2dTranspose(ngf * 2, ngf, 4, 2, 'pad', 1, weight_init=weight_init),
14    nn.BatchNorm2d(ngf, gamma_init=gamma_init),
15    nn.ReLU(),
16    nn.Conv2dTranspose(ngf, nc, 4, 2, 'pad', 1, weight_init=weight_init),
17    nn.Tanh()
18 )
```

判别器 (D)

判别器的核心功能是区分输入样本是真实图像还是生成器生成的伪造图像，本质上是一个二分类的卷积神经网络。DCGAN判别器采用“卷积+批量归一化+LeakyReLU激活”的组合结构

代码块

```
1 self.discriminator = nn.SequentialCell(
2     nn.Conv2d(nc, ndf, 4, 2, 'pad', 1, weight_init=weight_init),
3     nn.LeakyReLU(0.2),
4     nn.Conv2d(ndf, ndf * 2, 4, 2, 'pad', 1, weight_init=weight_init),
5     nn.BatchNorm2d(ndf * 2, gamma_init=gamma_init),
6     nn.LeakyReLU(0.2),
7     nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 'pad', 1, weight_init=weight_init),
8     nn.BatchNorm2d(ndf * 4, gamma_init=gamma_init),
9     nn.LeakyReLU(0.2),
10    nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 'pad', 1, weight_init=weight_init),
11    nn.BatchNorm2d(ndf * 8, gamma_init=gamma_init),
12    nn.LeakyReLU(0.2),
13    nn.Conv2d(ndf * 8, 1, 4, 1, 'valid', weight_init=weight_init),
14    )
```

3. 损失函数与优化器配置

这里使用的是 `BCELoss`（二元交叉熵损失），是针对**二分类任务**设计的损失函数，完全适配 DCGAN 中判别器的任务定位（二分类问题）。

然后使用了 `Adam` 优化器，相比 SGD 具有收敛速度更快、梯度更新更平稳的优势，非常适配 GAN 模型的不稳定训练特性。

代码块

```
1 # 定义损失函数
2 adversarial_loss = nn.BCELoss(reduction='mean')
3 # 为生成器和判别器设置优化器
4 optimizer_D = nn.Adam(discriminator.trainable_params(), learning_rate=lr,
5     beta1=beta1)
6 optimizer_G = nn.Adam(generator.trainable_params(), learning_rate=lr,
7     beta1=beta1)
```

4. 模型训练

这里模型训练其实就比较经典的GAN框架：生成器生成样本，然后拿判别器计算损失；判别器对样本进行分类，训练分类能力。

然后生成器和判别器交替训练。

```

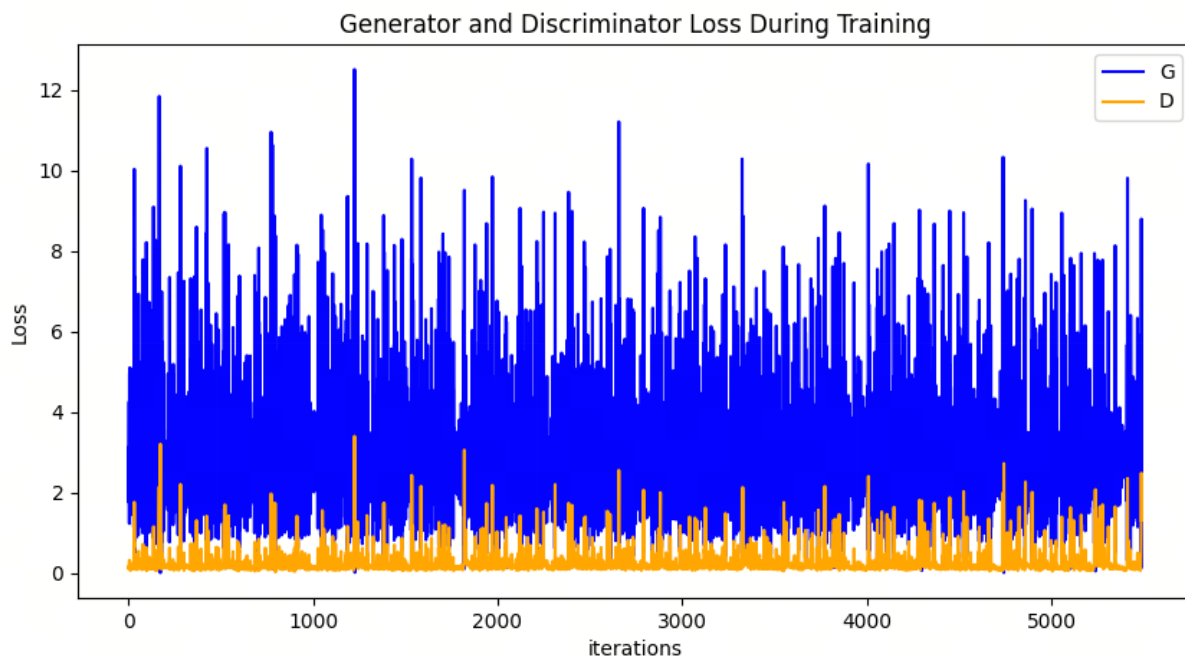
1 代码块 def generator_forward(real_imgs, valid):
2      # 将噪声采样为发生器的输入
3      z = ops.standard_normal((real_imgs.shape[0], nz, 1, 1))
4      # 生成一批图像
5      gen_imgs = generator(z)
6      # 损失衡量发生器绕过判别器的能力
7      g_loss = adversarial_loss(discriminator(gen_imgs), valid)
8      return g_loss, gen_imgs
9
10 def discriminator_forward(real_imgs, gen_imgs, valid, fake):
11     # 衡量鉴别器从生成的样本中对真实样本进行分类的能力
12     real_loss = adversarial_loss(discriminator(real_imgs), valid)
13     fake_loss = adversarial_loss(discriminator(gen_imgs), fake)
14     d_loss = (real_loss + fake_loss) / 2
15     return d_loss
16
17 grad_generator_fn = ms.value_and_grad(generator_forward, None,
18                                     optimizer_G.parameters,
19                                     has_aux=True)
20 grad_discriminator_fn = ms.value_and_grad(discriminator_forward, None,
21                                     optimizer_D.parameters)
22
23 @ms.jit
24 def train_step(imgs):
25     valid = ops.ones((imgs.shape[0], 1), mindspore.float32)
26     fake = ops.zeros((imgs.shape[0], 1), mindspore.float32)
27     (g_loss, gen_imgs), g_grads = grad_generator_fn(imgs, valid)
28     optimizer_G(g_grads)
29     d_loss, d_grads = grad_discriminator_fn(imgs, gen_imgs, valid, fake)
30     optimizer_D(d_grads)
31
32     return g_loss, d_loss, gen_imgs

```

四、可视化展示及结果生成

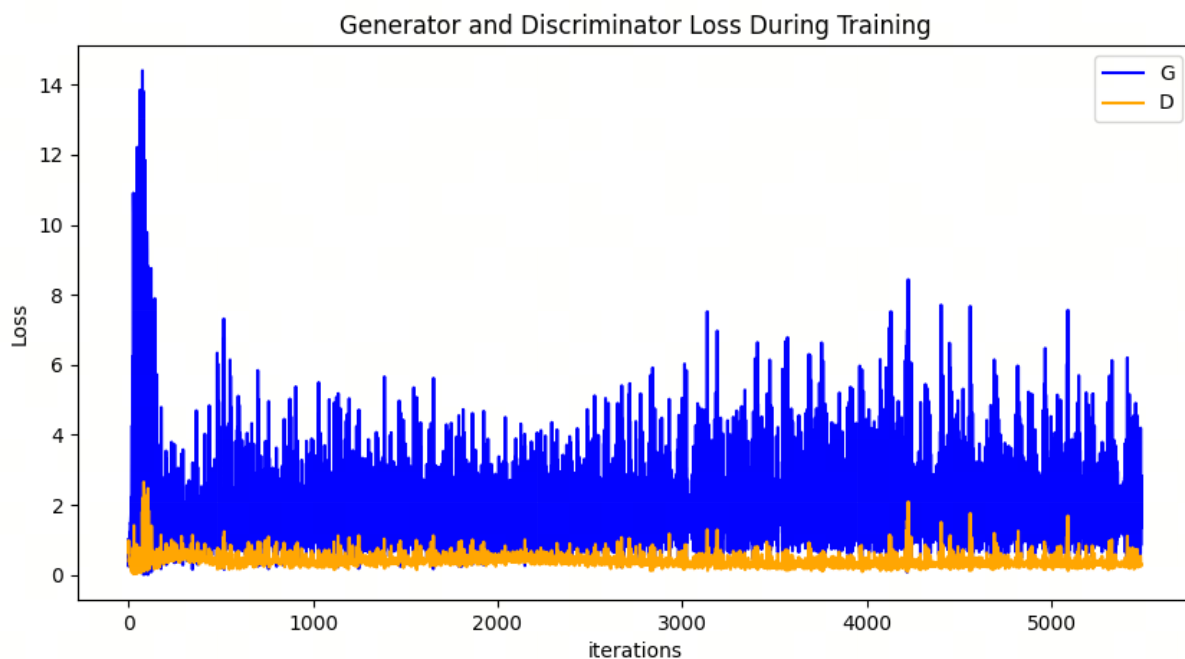
1. 损失函数收敛展示

下面这个是我一开始跑的，发现判别器过强导致生成器一致无法收敛，波动很大。

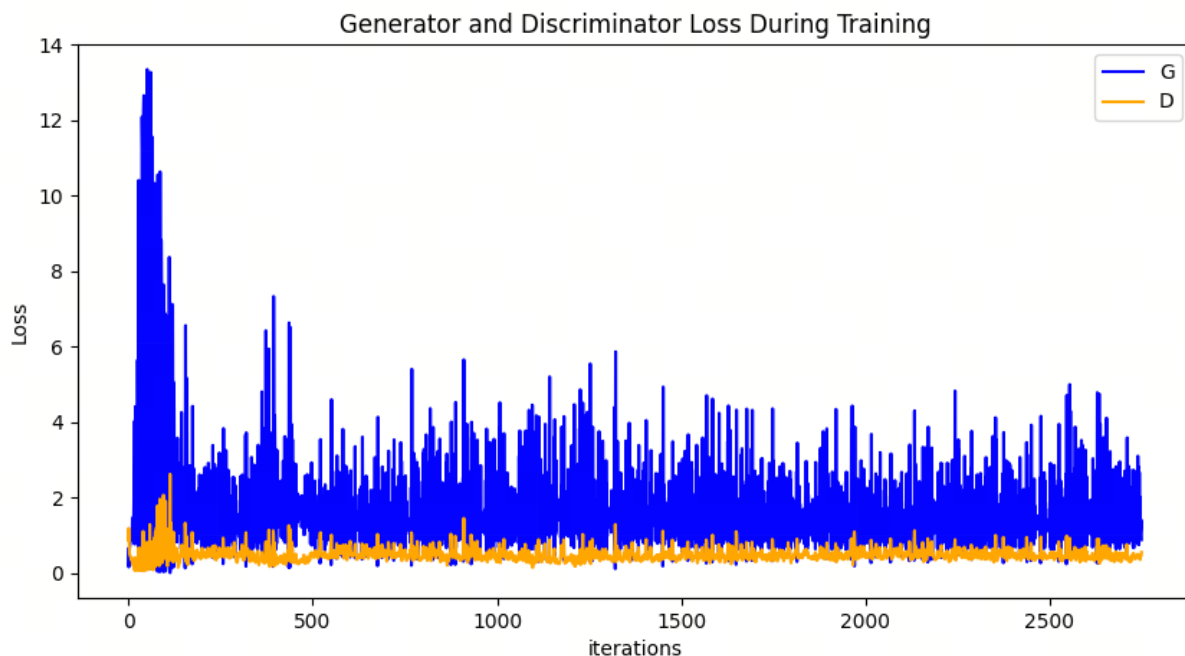


收敛失败情况

于是我尝试调整两个的**学习率**，原本两个的学习率都为 `0.0002`，后面我修改判别器的学习率为 `0.0001` 这样一开始判别器不会过强而导致生成器无法收敛，对比上面的稳定的多，但是仍然有波动。



由于一直波动，觉得可能是batchsize太小的原因，我这里还把batch增大到256大小，果然波动小了：



2. 训练过程可视化

下面这个是可视化训练过程中通过隐向量`fixed\_noise`生成的图像，可以看到能力越来越好。（但是似乎pdf导出不支持gif图）从左到右看，从上到下。



3. 图像生成展示

最终我们输入一组高斯噪声给生成器，得到一下图像，但是可以看到模糊还是很严重的，感觉这里需要增加模型的参数大小，但是实验进行到这里，留给以后学习吧。



五、实验小结

本次实验基于 MindSpore 框架成功搭建了经典深度卷积生成对抗网络（DCGAN），并以动漫头像数据集为训练对象，完整走完了「数据处理→网络构建→损失与优化器配置→模型训练→图像生成与可视化」的深度学习任务全流程，达到了预期的实验目的，同时也积累了 GAN 模型训练的实战经验。

实验过程中，基本可以按照指导书上面的代码进行运行，但是看结果并不太好，这里自己进行了调整，虽然后面生成器收敛效果好了很多，但是生成的图像仍然带有模糊，感觉是生成器的模型深度和参数大小导致的，无法去掉模糊，甚至带有伪影，后续也可以考虑在生成器末端用超分或者滤波器增强。